LedgerPilot

Technical Documentation

2026 FinTechathon · WeBank FinTech College Students Challenge — International Track

Topic E — SME Finance Copilot · v1.0 · October 2026

LedgerPilot is an AI agent that acts as a finance copilot for small-business owners. This document describes its system architecture, the core algorithms behind each capability, the data model, and the security design. A companion *Security Self-Assessment* covers the permission model and known-risk list in depth.

1. Problem & scope

Small and medium enterprises (SMEs) rarely have a finance team. Owners juggle operations while cash-flow problems build silently, financing decisions are made with incomplete information, and the cost of a bad borrowing choice is high. LedgerPilot addresses three concrete pain points from Topic E:

- **Cash-flow management with alerts** — know early when money will get tight.
- **Financing-option matching** — find the right instrument, with reasons.
- **Business financial analysis** — understand the health of the business at a glance.

The system is built as an **agent**: it reasons about which capabilities to invoke for a given request, executes them, and can take follow-up actions — subject to a permission model that keeps a human in control of anything that changes state.

2. System architecture

LedgerPilot follows a four-stage agent loop — **plan → gate → execute → narrate** — over a registry of typed tools, with every step written to an append-only audit log.



2.1 Components

Module	Responsibility
`agent/core.py`	The agent loop; orchestrates planning, gating, tool execution, and narration. Hosts the `AuditLogger`.
`agent/llm.py`	LLM adapter with two interchangeable backends (live API / deterministic offline). Exposes `plan()` and `narrate()`.
`agent/permissions.py`	The three-tier permission model and confirmation logic.
`agent/datastore.py`	Read-only access to the sandbox data; cached loads.
`tools/`	Four tools, each declaring a `REQUIRED_TIER` and a JSON `SCHEMA` for LLM tool-calling.
`ui/`	Flask app and single-page chat interface with a live business snapshot.
`data/`	Business profiles, financing-product catalog, deterministic data generator.

2.2 The dual LLM backend

The adapter speaks one small interface regardless of backend:

```
plan(user_msg, context)             -> list[ToolCall]
narrate(user_msg, tool_results, ctx) -> str
```

- **API backend** — any OpenAI-compatible chat-completions endpoint with native tool-calling. Activated when `LEDGERPILOT_API_KEY` is set. Used for the live demo.
- **Offline backend** — a deterministic intent router (regex-based intent detection + entity extraction for business, amount, and purpose) followed by a templating narrator that verbalises structured tool output. Requires no network or key.

> The offline backend is a deliberate engineering choice, not a stub. It lets the **entire agent loop — including permission gating and audit logging — run reproducibly in any sandbox**, which is how the submitted execution evidence is generated. Flipping on an API key upgrades the reasoning quality without changing tools, gating, or logging.

3. Core algorithms

3.1 Cash-flow analysis & proactive alerts (analyze_cashflow, T0)

Transactions are bucketed by month into inflow/outflow; the engine then computes net position, a runway estimate, a 30-day projection, and a set of alerts:

- **Runway** — when average monthly net is negative: `runway_months = cash_on_hand / |avg_monthly_net|`. Flagged **high** below 3 months, **medium** below 6.
- **Thin-buffer alert** — fires when `cash_on_hand < 0.5 × avg_monthly_outflow`.
- **Expense-spike detection** — per category, the latest month is compared against that category's own history using a 2-sigma rule: `current > mean + 2·stdev` *and* `current > 1.3 × mean` (the second condition suppresses noise on low-variance categories).
- **30-day projection** — `cash + avg_net + 0.5 × receivables_due_within_30d` (a conservative half-collection assumption on near-term invoices).
- **Overdue-receivable alert** — totals invoices past due and surfaces them as the fastest available cash win.

3.2 Financial-health analysis (financial_analysis, T0)

Core metrics are derived from the six-month transaction window: net margin, expense ratio, debt-to-annual-revenue, revenue trend (first-half vs second-half average), and the top three cost drivers by share of spend.

These feed an **explainable 0–100 health score** built from four normalised components, each independently interpretable:

Component	Normalisation	Weight
Margin	`min(100, net_margin% × 4)` — 25% margin → 100	30%
Liquidity	`min(100, cash / outflow × 50)` — 2 months cash → 100	25%
Leverage	`100 − debt_to_annual_revenue × 100` — debt-free → 100	25%
Growth	`50 + revenue_trend% × 2.5` — flat → 50	20%

The final score is the weighted sum, mapped to a band (Strong ≥75, Stable ≥55, Watch ≥40, else At risk). Because the components and weights are returned alongside the score, every number is defensible — a requirement for financial explainability and compliance.

3.3 Financing matcher (match_financing, T1)

A two-stage, fully rule-based matcher over a catalog of six financing product types (term loan, invoice financing, credit line, revenue-based, government MSME scheme, merchant cash advance):

1. **Hard eligibility** — credit-band rank, minimum business age, and amount caps. Failures are recorded *with reasons* and reported as "not yet eligible".
2. **Fit scoring** — eligible products start at 50 and are adjusted by signals derived from the business: open-receivables ratio (favours invoice financing), recurring tight months (favours a revolving line), margin (gates revenue-based), GST registration (favours low-cost government schemes), positive net cash flow (supports term-loan EMIs), plus purpose nudges from the user's stated intent. Merchant cash advance is penalised as a last resort.

Each recommendation carries its fit score, APR range, an estimated annual interest cost, the explicit reasons it fits, and its key requirement. The matcher never issues a credit decision — only explainable guidance with a clear disclaimer.

3.4 Action scheduling (schedule_action, T2)

The one state-changing tool. It writes a reminder/alert to an actions ledger (in production: fires a real notification). It can never execute without an explicit, matching user confirmation — see §5.

4. Data model

The sandbox mirrors what a real deployment would read from an accounting integration (e.g. Tally, Zoho Books, or bank statements), so swapping in a live data source is a datastore change, not an algorithm change.

Entity	Key fields
Business	id, sector, currency, monthly revenue/expense averages, cash on hand, outstanding debt, credit-score band, GST status, account age
Transaction	txn_id, date, type (credit/debit), category, amount
Invoice	invoice_id, client, issued/due dates, amount, status (open/overdue), days-to-due
Financing product	type, provider, amount range, APR range, tenure, minimum credit band & business age, collateral, key requirement

Data is generated deterministically (fixed seed) so that execution evidence is byte-for-byte reproducible across runs.

5. Security design (summary)

Security is treated as a first-class design concern, not a wrapper. The four pillars:

- **Permission tiers** — every tool declares the minimum tier it needs (T0 read, T1 recommend, T2 act). The agent runs under a configurable ceiling.
- **Confirmation-always for actions** — a T2 tool raises `ConfirmationRequired` unless the user has explicitly confirmed the exact pending action. The agent never self-approves.
- **Full auditability** — every user message, plan, tool call (with its required tier), result, confirmation prompt, and denial is appended to a JSONL audit log.
- **Explainability & non-advice** — recommendations always carry their reasons and a disclaimer that they are not a credit offer or regulated financial advice.

Full detail, including the known-risk list and mitigations, is in the *Security Self-Assessment* deliverable.

6. Deployment

```
pip install -r requirements.txt
python ledgerpilot/data/generate_data.py   # deterministic sandbox data
python -m ledgerpilot.ui.app               # web app at :5000
python run_scenario.py                     # regenerate execution evidence
pytest -q                                  # 12 tests
```

Offline mode needs only Flask. The live LLM backend is enabled by setting `LEDGERPILOT_API_KEY` (any OpenAI-compatible provider) in `.env`. The same code path serves both; the active engine is shown in the UI header.

7. Limitations & roadmap

- Sandbox data stands in for live accounting/bank integrations; the datastore interface is the single seam to replace.
- The financing catalog is representative, not exhaustive, and APRs are indicative ranges rather than live lender quotes.
- Next steps: live accounting-software connectors, lender-API integration for real quotes, multi-currency support for cross-border SMEs, and a scheduled background pass that pushes alerts proactively between sessions.
